Modelling Software-based Systems:
Correct by Construction Paradigm
Chapter *The Modelling Language Event-B*
Version November 9, 2026 at 2:03pm

Dominique MÉRY
2:03pm

November 9, 2026

Contents

Chapter 1. Design of Correct by Construction Sequential Algorithms 1
Dominique MÉRY

1.1. Introduction 1
1.2. Design of an Iterative Sequential Algorithm 4
1.2.1. Problem 1: Calculating the sum of a vector of integer values . . . 4
1.2.1.1. Specification of the problem to solve 6
1.2.1.2. Refining to compute inductively 6
1.2.1.3. Focus on the value to be preserved 7
1.2.1.4. Obtaining an algorithmic machine 8
1.2.1.5. Comments on the methodology 10
1.2.2. Transforming Event-Bmachines into sequential algorithms 10
1.3. Examples of development 12
1.3.1. Problem 2: Computing the function cube $\lambda x.x^3$ using only additions 12
1.3.2. Problem 3: Searching for a value in an array 16
1.3.3. Problem 4: Computing a primitive recursive function 18
1.4. Design of a Recursive Sequential Algorithm 20
1.4.1. The "Call as Event" Idea 20
1.4.2. Applying the call as event technique 24
1.4.2.1. Problem 5: Computing the square $(\lambda x.x^2)$ 24
1.4.2.2. Problem 6: Binary search in an array 27
1.4.3. Comments on the call as event idea 31
1.5. Final comments 32

1

Design of Correct by Construction Sequential Algorithms

Dominique MÉRY
[1]*Université de Lorraine, LORIA, France*

1.1. Introduction

The development of correct algorithms or sequential programs from specifications (**?**) is a scientific theme linked to that of the verification of programs or algorithms (**???**). The fundamental question can be summarised in the form of a symbolic equation $\mathsf{D}, \mathsf{A} \Rightarrow \mathsf{C}$ where D (resp. A, C) is the problem domain (resp. the algorithm, the contract). In this equation, we assume that the problem domain D is known and may be, for example, $\mathbb{Z}$, the domain of integers, and we will be interested in problems requiring properties on integers. A problem is a general expression to designate the calculation of a value from data or the search for a value in a set of data. The algorithm A expresses assignment instructions, conditional instructions and bounded or unbounded iterations. Finally, the contract C is defined by two elements: a pre-condition $\mathsf{pre}(v_0)$ and a post-condition $\mathsf{post}(v_0, v_f)$ relating the initial value v_0 of a variable v to its final value v_f. Solving the problem consists in expressing it by a contract ensuring that for any initial value v_0 satisfying $\mathsf{pre}(v_0)$, there exists a value v_f satisfying $\mathsf{post}(v_0, v_f)$. On the other hand, it is important that the final value v_f corresponds to a calculation of an algorithm A in the classical sense of computability (**?**). The equation can therefore be rewritten in the following form: $\forall v_0, v_f \in \mathsf{D}.\mathsf{pre}(v_0) \wedge v_0 \xrightarrow{\mathrm{A}} v_f \Rightarrow \mathsf{post}(v_0, v_f)$

Design of Correct by Construction Sequential Algorithms,
coordinated by Dominique MÉRY. © ISTE Editions 2019.

and we obtain the expression for the partial correctness of the algorithm A in relation to the contract $\mathsf{C}(v, \mathsf{pre}(v_0), \mathsf{post}(v_0, v_f))$ on the domain D. The relation $\xrightarrow{\mathrm{A}}$ expresses the calculation of A and we can add a second expression which plays the role of the termination of A, assuming that A is deterministic (otherwise, termination requires that *every* computation from v_0 is finite): $\forall v_0 \in \mathsf{D}.\mathsf{pre}(v_0) \Rightarrow \exists v_f \in \mathsf{D}.v_0 \xrightarrow{\mathrm{A}} v_f$. The two translations produce a synthetic expression of the following form:

$$\forall v_0 \in \mathsf{D}.\mathsf{pre}(v_0) \Rightarrow \begin{pmatrix} \forall v_f \in \mathsf{D}.v_0 \xrightarrow{\mathrm{A}} v_f \Rightarrow \mathsf{post}(v_0, v_f) \\ \exists v_f \in \mathsf{D}.v_0 \xrightarrow{\mathrm{A}} v_f \end{pmatrix}$$

which we rewrite with the wp calculus as follows: $\forall v_0 \in \mathsf{D}.v = v_0 \wedge \mathsf{pre}(v_0) \Rightarrow wp(\mathsf{A})(\mathsf{post}(v_0, v))$

This discussion leads us to give meaning to the correctness of an algorithm by considering its partial correctness as well as its termination. Hoare logic most often expresses partial correctness and in all rigour it would be necessary to use two notations, one expressing partial correctness and the other total correctness, but the objective here is not to verify an algorithm A and therefore to verify a list of verification conditions as Floyd's method indicates, but to find an algorithm A which satisfies this equation $\forall v_0 \in \mathsf{D}.v = v_0 \wedge \mathsf{pre}(v_0) \Rightarrow wp(\mathsf{A})(\mathsf{post}(v_0, v))$. In the *a posteriori* approach to the correctness of algorithms, we propose a solution for A and then check a list of verification conditions. This technique also consists of checking the verification conditions without having clearly stated the contract C.

Semantic analysis techniques, such as abstract interpretation (**?**), can also be used: they provide the programmer with analysis feedback. Correctness by construction is a technique which starts with an abstract algorithm A_0 which fulfils the contract in a verified way and which is progressively enriched with increasingly complex control structures, while preserving the property of correctness with respect to the contract C. This progressive strategy of adding elements to make the result more precise is guided by the refinement between the algorithms. Each transformation or refinement step guarantees that the resulting algorithm is correct. C. Morgan (**?**) develops the refinement calculus which makes it possible to progressively and correctly transform one algorithm into another algorithm. The operation guarantees that the final algorithm is correct with respect to the first algorithm which is a pre/post specification considered as an algorithmic statement of the form $v : [\mathsf{pre}, \mathsf{post}]$, whose effect is to modify v by respecting the contract defined by pre and post. Thus, the strategy consists of constructing a sequence of algorithms $\mathsf{A}_0, \ldots, \mathsf{A}_i, \ldots, \mathsf{A}_n$ with the properties:

– A_0 is the expression of the contract: $\mathsf{D}, \mathsf{A}_0 \Rightarrow \mathsf{C}$

– for all i in $1..n$, the algorithm A_i refines A_{i-1} ($\mathsf{A}_{i-1} \sqsubseteq \mathsf{A}_i$), so that $\mathsf{D}, \mathsf{A}_{i-1} \Rightarrow \mathsf{C}$ implies $\mathsf{D}, \mathsf{A}_i \Rightarrow \mathsf{C}$;

– consequently, A_n is an algorithm satisfying the contract: $\mathsf{D}, \mathsf{A}_n \Rightarrow \mathsf{C}$.

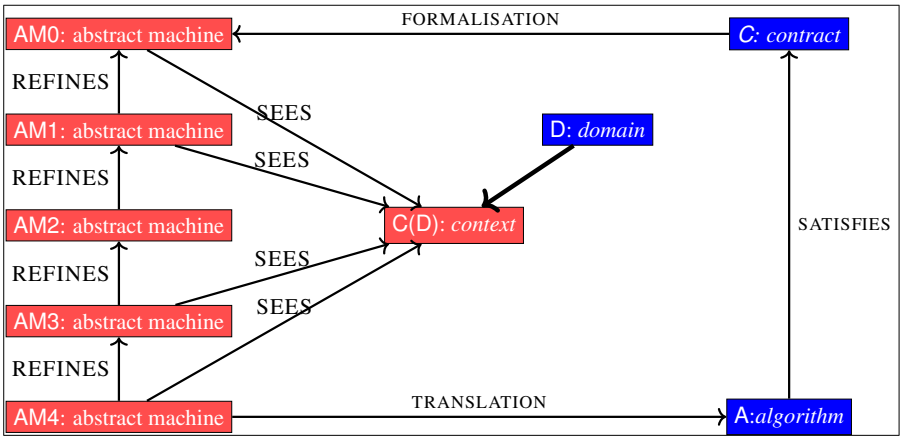


Figure 1.1. *Correctness by construction in Event-B*

More recently, Derrick G. Kourie and Bruce W. Watson (**?**) follow this strategy and implement the correctness-by-construction paradigm on classical programming problems. In both approaches, the contract becomes an algorithmic statement corresponding to a generalised algorithmic structure. In fact, as we can see, a contract can express the halting problem of Turing machines (**?**) in a language of assertions which is still abstract, which does not mean that we have solved the halting problem, but that we have extended the algorithmic language with a statement *magic* enabling it to be solved. This amounts to extending the space of solutions and then choosing what corresponds to the theory of computability. C. Morgan reminded us that all second-degree equations have solutions in the field of complex numbers, but that the method of solving in the set of reals retains only the real solutions. The specification statement $v : [\mathsf{pre}, \mathsf{post}]$ is a valid statement in this algorithmic language. Such a specification instruction can be expressed in the Event-Blanguage.

This approach to developing correct by construction algorithms is quite simply implemented in the Event-Blanguage and supported by the Rodin environment. Figure 1.1 describes the general idea. This idea consists of translating the contract as an *event* which performs the calculation described by the contract. The contract expresses the *what*, and the computation is observed through the event. This event is placed in a first abstract machine AM0, which uses the mathematical elements extracted from the problem domain D and expressed in the Event-Bcontext C(D). The problem domain D is a structure with data and properties related to the problem to solve and it is generally the set of natural numbers or integers for arithmetic, or arrays for searching or sorting. C(D) is a context designed from the problem domain by adding axioms and theorems

for sets and constants that are necessary for the derivation of Event-Bmachines. The development of an algorithm consists in the gradual enrichment of the abstract machines AM0, ..., AMn, by expressing the computations necessary to systematically translate the last machine into an algorithmic form so as to guarantee the correctness of the algorithm A, thanks to the correctness of the transformations provided by the refinement. It should be noted that correctness concerns partial correctness and termination, and that the abstract machines are models containing variables that will not be necessarily present in the algorithm produced.

We will present two techniques that implement this development method:

– the inductive pattern based on transformations of abstract machines by Jean-Raymond Abrial (**?**, Chapter 15).

– the recursive pattern based on the relation *call* as *event* that we have developed (**??**).

We will present these two methods by highlighting case studies of classical sequential algorithms.

1.2. Design of an Iterative Sequential Algorithm

1.2.1. *Problem 1: Calculating the sum of a vector of integer values*

First, we define the contract associated with this problem of calculating the sum of the elements of the vector v_0. The algorithm we are looking for is SUM.

The domain of the problem to be solved is that of sequences of integers $\mathbb{SEQ}(\mathbb{Z})$, and the contract states that the value of the result is the sum of the integers in the sequence v_0. This mathematical expression is not directly expressible in the mathematical language of Event-Band we define a sequence u characterising the values of the partial sums. The context *S0* of our Event-Bmodel is defined by enumerating the *requires* hypotheses and defining u.

First, we need to express the sum r of the sequence v_0 in the language of Event-B; this formulation is immediate in mathematical terms: $r = \sum_{k=1}^{k=n_0} v_0(k)$. As the notation for summing a finite sequence of values is not provided in the basic elements of the language, we must *define* this notion in a context *S0* which will contain the data of the problem and the notations defined specifically for this case.

variables n, v, r

requires
$$\begin{cases} n_0 \in \mathbb{N} \land n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \end{cases}$$

ensures
$$\begin{cases} r_f = \sum_{k=1}^{k=n_0} v_0(k) \\ n_f = n_0 \\ v_f = v_0 \end{cases}$$

Thus, the *data* n_0 and v_0 are defined as being respectively a non-zero natural integer (axioms pre1, pre2) and a function v_0 of domain $1..n_0$ and codomain $\mathbb{Z}$ (axiom pre3). The aim is to define the theory in which we will describe our data.
Naming the axioms with the prefix *pre* declares the first three axioms as facts given by the precondition. From a programming viewpoint, they define the contract signature, i.e., the types of the input parameters of the function to be implemented.

Secondly, we introduce a sequence u of integer values corresponding to the partial sums $\sum_{k=1}^{k=i} v_0(k)$. To do this, the idea is to define the partial sums using an inductive definition, which technically requires us to be sure of the *well-definedness* of this sequence u. The sequence u is therefore defined as follows:

– u is a total function from $0..n_0$ to $\mathbb{Z}$ (axiom axm1).

– Initially, the summation starts with 0 and $u(0) = 0$ (axiom axm2).

– For values of k in $1..n_0$, the value of $u(k)$ is defined from that of $u(k-1)$ and $v_0(k)$ (axiom axm3).

Axioms are given in the context *S0* and constitute a logical and mathematical theory, which will be useful for proving the properties of the models we will develop later.

CONTEXT $S0$
CONSTANTS $n0\ v0\ u$
AXIOMS
$@pre1\ n0 \in \mathbb{N}$
$@pre2\ n0 \neq 0$
$@pre3\ v0 \in 1..n0 \rightarrow \mathbb{Z}$
$@axm1\ u \in 0..n0 \rightarrow \mathbb{Z}$
$@axm2\ u(0) = 0$
$@axm3\ \forall k.\ k \in 1..n0 \Rightarrow u(k) = u(k-1) + v0(k)$
END

Proof obligations (WD) ensure the well-definedness of axioms. We have therefore defined the mathematical framework of the problem and we will now define the problem of summing the sequence v_0.

1.2.1.1. *Specification of the problem to solve*

The problem is therefore to calculate the sum of the sequence v. We define a machine *S1* which is an abstract machine expressing, through the event final, the *post-condition* $r = u(n)$. In fact, the new value of the variable r will be $u(n)$, when the event final is observed. The initial value of r is arbitrary at initialisation.

MACHINE $S1$ SEES $S0$
VARIABLES $r\ v\ n$
INVARIANTS
$@inv1\ r \in \mathbb{Z}$
$@inv2\ n \in \mathbb{Z}$
$@inv3\ v \in 1..n \rightarrow \mathbb{Z}$
$@read-values\ n = n0 \ \wedge \ v = v0$
EVENTS
EVENT $INITIALISATION$
then
$@act1\ r :\in \mathbb{Z}$
$@act2\ n := n0$
$@act3\ v := v0$
end
EVENT $final$
then
$@act1\ r := u(n)$
end
$anticipated$ EVENT $keep$
then
$@act1\ r, n, v :|$
$(\ r' \in \mathbb{Z} \wedge n' \in \mathbb{Z}$
$\wedge\ v' \in 1..n' \rightarrow \mathbb{Z}$
$\wedge\ n' = n0 \ \wedge \ v' = v0\ )$
end
END

Finally, the variable r must satisfy the invariant $inv1 : r \in \mathbb{Z}$ which specifies the type of r. The event final is therefore simply an assignment of the value $u(n)$ to r.
We can express it as a Hoare triple:
$\{n = n_0 \wedge v = v_0 \wedge n_0 > 0 \wedge v_0 \in 1\,..\,n_0 \rightarrow \mathbb{Z}\}\mathsf{SUM}\{r = u(n_0)\}$.

Note that the data are *visible* (sees) from the context *S0*. The problem is therefore to find an algorithm that calculates the value $u(n)$ and stores it in r.
A second event called keep can also be added to simulate some hidden activity before the observation of the event final. Note that the event is *anticipated* which means that it hides a *ghost* event which will appear later explicitly and effectively. It states that *something like a loop* is acting until the event final.
We have therefore described the problem domain to be solved and we have formulated what we want to calculate. The next step is to invent a *method of calculation*, which requires an *idea of a solution* and the use of refinement.

1.2.1.2. *Refining to compute inductively*

We have defined the specification of the problem of calculating the sum of the elements of a sequence v_0 and we need to find a way to *calculate* the value of the sequence u at term n_0 corresponding to the summation of v_0. The assignment $r := u(n_0)$ is an expression mixing a variable r and a mathematical value $u(n_0)$. A trivial and inefficient solution is well known: store the values of the sequence u in an array uu and translate the assignment into the form $r := uu(n)$ where uu verifies the following

property $\forall k.k \in dom(uu) \Rightarrow uu(k) = u(k)$ and this property constitutes an element of the invariant inv4. The idea is therefore to use the variable uu ($uu \in 0 \,..\, n_0 \pfun \mathbb{Z}$) to control the calculation and its progress. Progression is ensured by the event step2, which decreases the quantity $n - i$ and therefore ensures that the process converges.

MACHINE $S2$
REFINES $S1$
SEES $S0$
VARIABLES $r\ i\ uu\ n\ v$
INVARIANTS
$@inv1\ i \in 0..n$
$@inv2\ uu \in 0..n \pfun \mathbb{Z}$
$@inv3\ dom(uu) = 0..i$
$@inv4\ \forall k.\ k \in dom(uu) \Rightarrow uu(k) = u(k)$
$theorem\ @inoutdata1\ v = v0$
$theorem\ @inputdata2\ n = n0$
VARIANT $n - i$

EVENT $INITIALISATION$
then
$@act1\ r :\in \mathbb{Z}$
$@act3\ i := 0$
$@act4\ uu := \{0 \mapsto 0\}$
$@act5\ n := n0$
$@act6\ v := v0$
end
EVENT $final2$
REFINES $final1$
where
$@grd1\ n \in dom(uu)$
then
$@act1\ r := uu(n)$
end
$convergent$ EVENT $step2$
REFINES $keep$
where
$@grd1\ n \notin dom(uu)$
then
$@act1\ i := i + 1$
$@act2\ uu(i+1) := uu(i) + v(i+1)$
end
end

The machine *S2* therefore describes a process which progressively fills the array uu and therefore retains all the intermediate results. Proof obligations are fairly easy to discharge insofar as we have *prepared* the work of the proof assistant. We will give the details of the statistics in a table at the end of the development. It is quite clear that the variable uu is in fact a witness or a trace of the intermediate values and that this variable can therefore be hidden in this model which will have to be refined. Before hiding this variable, we will set aside the value $uu(i)$ that we need to keep.

1.2.1.3. *Focus on the value to be preserved*

The following refinement S3 will lead to the introduction of a new variable cu which will retain the last current value $uu(i)$. We therefore operate a *superposition* (**?**)

on the machine S2. This machine refines (simulates) the machine S2; hence the properties of S2 remain satisfied by the new machine S3, provided that all proof obligations are discharged.

MACHINE $S3$
REFINES $S2$
SEES $S0$
VARIABLES $r\ i\ uu\ cu\ n\ v$
INVARIANTS
$@inv1\ cu \in \mathbb{Z}$
$@inv2\ cu = uu(i)$
EVENT $INIT$
then
$@act1\ r :\in \mathbb{Z}$
$@act2\ i := 0$
$@act3\ uu := \{0 \mapsto 0\}$
$@act4\ cu := 0$
$@act6\ n := n0$
$@act7\ v := v0$
end

EVENT $final3$
REFINES $final2$
where
$@grd1\ i = n$
then
$@act1\ r := cu$
end

EVENT $step3$
REFINES $step2$
where
$@grd1\ i < n$
then
$@act1\ i := i + 1$
$@act2\ uu(i+1) := uu(i) + v(i+1)$
$@act3\ cu := cu + v(i+1)$
end
end

The machine S3 is very expressive and provides details about the information required to ensure that it is suitable for the problem expressed in the machine S1, which is refined by the machine S2. However, the machine S3 is expensive in terms of variables and the refinement allows us to leave only the variables that are useful for the calculation. In what follows, we will make the machine *more algorithmic* and retain only those variables in the concrete machine that are sufficient for the calculation.

1.2.1.4. *Obtaining an algorithmic machine*

In this final step, we refine the machine S3 into a machine S4 and hide the variable uu of the abstract machine S3. Thus, the machine S4 includes variables r, n, v, cu and i and we will also note that it satisfies safety properties, called theorems in Event-B. These properties are proved from the properties of previous refined machines. We have thus obtained a machine comprising an initialisation and two events:

– The event final4 is observed when the value of i is n and, in this case, the variable cu contains the value $u(n)$. The invariant guarantees that the value of cu is $u(n)$.

– The event step4 is observed when the value of i is less than n. This also means that, as long as this value is less than n, the event can be observed and the state transitions generated from these events therefore correspond to *small* steps of an iterative algorithmic structure.

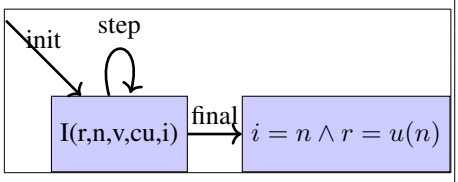
MACHINE $S4$
REFINES $S3$
SEES $S0$
VARIABLES $r\ i\ cu\ n\ v$
INVARIANTS
$theorem\ @inv1\ cu = u(i)$

EVENTS
EVENT $INITIALISATION$
then
$@act1\ r :\in \mathbb{Z}$
$@act2\ i := 0$
$@act4\ cu := 0$
$@act5\ v := v0$
$@act6\ n := n0$
end

EVENT $final4$
REFINES $final3$
where
$@grd1\ \neg(i < n)$
then
$@act1\ r := cu$
end

EVENT $step4$
REFINES $step3$
where
$@grd1\ i < n$
then
$@act1\ i := i + 1$
$@act3\ cu := cu + v(i + 1)$
end
end

The Rodin project archive abk-summation corresponds to this development by refinement, taking care to use the calculation method defined by the sequence u. On the left, the following figure describes a view of events observed as labels of transitions from a node containing the loop invariant denoted $I(r, n, v, cu, i)$ and a node containing the postcondition of the contract. The right part contains the algorithm generated by applying rules on events of the machine *S4*.



$r :\in \mathbb{Z} \parallel i := 0$
$\parallel cu := 0 \parallel v := v0$
$\parallel n := n0$
while $i < n$ do
$i := i + 1 \parallel cu := cu + v(i + 1)$
od;
$r := cu$;

The components of abk-summation are constructed using the sequence u as a methodological guide, taking care to obtain conditions that can be expressed in an algorithmic language. In our case, the condition $n \notin dom(uu)$ (resp. $n \in dom(uu)$) is refined by the condition $i < n$ (resp. $i = n$). Note that the diagram on the left

corresponds to the algorithm on the right. These transformations can be defined more precisely and are implemented in the EB2ALGO (**?**) plugin which produces the above algorithm from the abk-summation archive. Jean-Raymond Abrial (**?**, Chapter 15) suggests progressive transformation rules to be applied to the events of machines like S4, and we give a more complete treatment of these transformation rules implemented in EB2ALGO (**?**).

1.2.1.5. *Comments on the methodology*

A specific methodology was employed in the selection of the variables. The variable uu is utilised for the storage of the calculated values of the sequence u, with the convention being to link the sequence u and the variable uu obtained by doubling the name of u. Obviously, we don't want to store all the intermediate values, just the ones used by the induction step. So the variable cu holds the value of uu that is used in the induction step. uu is a model variable that is not needed by the algorithm. However, it makes the proofs easier, so it is kept in the abstract machines S2 and S3 and hidden only in S4. Hiding uu provides a truly algorithmic view. It is also possible to obtain the termination of this algorithm with minimal effort, thanks to the variant which indicates that the event step4 leads to the convergence of this algorithm. The name final is imposed by the plugin EB2ALGO (**?**), following the conventions of Jean-Raymond Abrial (**?**, Chapter 15).

1.2.2. *Transforming Event-Bmachines into sequential algorithms*

We draw the conclusions of this simple problem and add the extra elements needed to develop iterative sequential algorithms. The plugin EB2ALGO implements the transformations of Jean-Raymond Abrial (**?**, Chapter 15). We apply two transformations to the abstract machine obtained at the end of the refinement process, once the model variables have been hidden. We recall the algorithmic language used by Jean-Raymond Abrial (**?**, Chapter 15).

Definition 1 *(The Pidgin Programming Language (**?**))*

Statements of the language are:

– $variable_list := expression_list$

– statement; statement

– *if condition* **then** *statement* **else** *statement* **end**

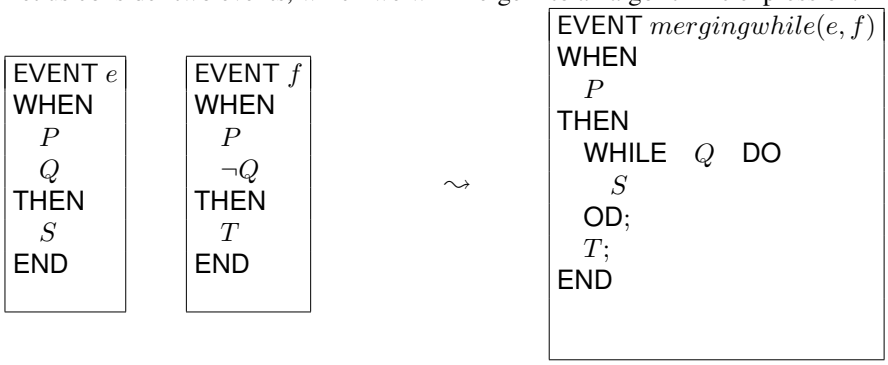
– *if condition* **then** *statement* **elseif** ... else *statement* **end**

– **while** *condition* **do** *statement* **end**

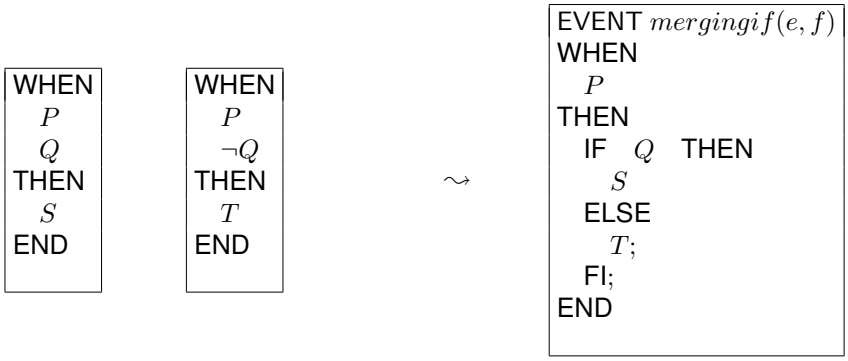
A program can be *broken down* into a set of events, which are then triggered according to the values of the variables. This decomposition leads to the use of a composition of events. The idea is straightforward, but it must adhere to strict conventions

to use the plugin EB2ALGO (**?**). We give these conditions which are implemented in the plugin and which must be satisfied during development design. Rules are given without proof of correctness and use a mixed language merging Event-Bevents and programming constructs.

Let us consider two events, which we will merge into an algorithmic expression:



We must specify the conditions of application. The event e appears as new or non-anticipated, therefore convergent at a lower level of refinement than that of f. We can be sure that there is a variant which terminates the loop. We must also assume that P is preserved by the event e. The event mergingwhile(e,f) appears at the same level as the event f. The guard P may be absent, which corresponds to $P = true$.



This transformation should be applied when the two events have been introduced at the same refinement level. The event mergingif(e,f) must appear at the same level as the events e and f. The guard P may be absent.

These two transformations can be used to design a plugin that produces a program in the language Pidgin. The first event is always called final and corresponds to the contract. Then the refinement process guides the design phase and it is also important to express that the new events that are introduced must decrease a variant that ensures

the convergence of the process described by the events. The new events are called convergent events and proof obligations are generated to handle the convergence of the events with respect to the variant.

Let us take the example we've already dealt with and apply the transformations.

MACHINE $S4$
REFINES $S3$
SEES $S0$
VARIABLES $r\ i\ cu\ n\ v$
INVARIANTS
$theorem\ @inv1\ cu = u(i)$

EVENTS
EVENT $INITIALISATION$
then
$@act1\ r :\in \mathbb{Z}$
$@act2\ i := 0$
$@act4\ cu := 0$
$@act5\ v := v0$
$@act6\ n := n0$
end

EVENT $final4$
REFINES $final3$
where
$@grd1\ \neg(i < n)$
then
$@act1\ r := cu$
end

EVENT $step4$
REFINES $step3$
where
$@grd1\ i < n$
then
$@act1\ i := i + 1$
$@act3\ cu := cu + v(i + 1)$
end
end

We have given an example of how to apply the merge transformation for iteration (see Fig. 1.2) and we refer the reader to the plugin EB2ALGO that implements these transformations. The next section illustrates the different ways to use the transformations of Event-Bmodels into the Pidgin programming language.

1.3. Examples of development

1.3.1. *Problem 2: Computing the function cube $\lambda x.x^3$ using only additions*

The objective is to calculate the function cube ($\lambda x.x^3$) of a non-negative integer using only addition operations. First, we define a sequence z corresponding to the cubes of non-negative integers. We do not explain how we found the sequence z, which satisfies the correctness property $\forall n \in \mathbb{N}.z(n) = n * n * n$. Moreover, the inductive definition of z gives a way to compute the term $z(n)$ for any natural number n. Hence, we have a computing method for the function cube using the sequence z.

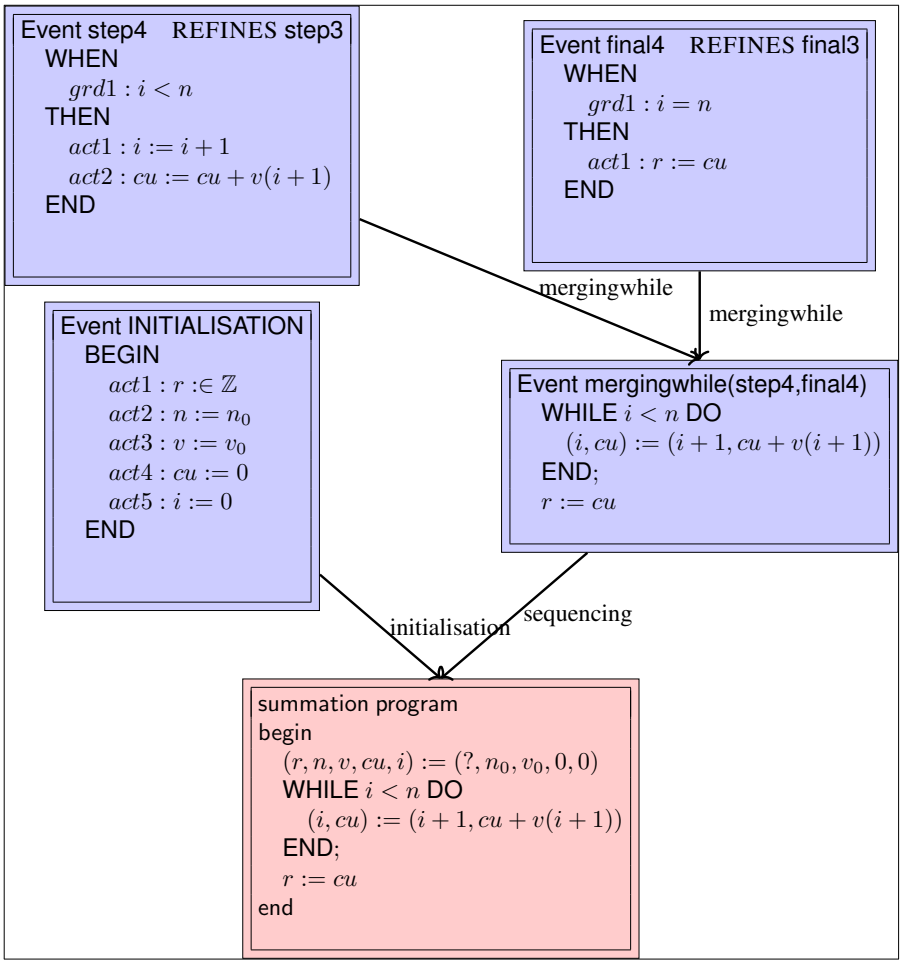


Figure 1.2. *Structure of applied transformations*

variables x, r

requires $x_0 \in \mathbb{N}$, $r_0 \in \mathbb{Z}$

ensures $r_f = x_0 * x_0 * x_0$, $x_f = x_0$

Calculating the function cube $(\lambda x.x^3)$ using only addition is based on the following sequences:

- $z_0 = 0$ and $\forall n \in \mathbb{N} : z_{n+1} = z_n + v_n + w_n$
- $v_0 = 0$ and $\forall n \in \mathbb{N} : v_{n+1} = v_n + t_n$
- $t_0 = 3$ and $\forall n \in \mathbb{N} : t_{n+1} = t_n + 6$
- $w_0 = 1$ and $\forall n \in \mathbb{N} : w_{n+1} = w_n + 3$
- $u_0 = 0$ and $\forall n \in \mathbb{N} : u_{n+1} = u_n + 1$

The first step is to show that the sequence z defines the sequence of cubes of non-negative integers and therefore gives an inductive way of calculating the cube of a non-negative integer x_0 using addition only. The domain of the problem to be solved is that of integers $\mathbb{Z}$ and the contract expresses that the value of the result is the cube of the non-negative integer x_0.

The context P3-0 expresses the sequences defining the values to be calculated to produce a value corresponding to the cube of x_0. The important result is the following theorem, proved with the Rodin platform.

Propery 1 *(Soundness of the sequence z)*

$$\forall k.k \in \mathbb{N} \Rightarrow z(k) = k * k * k$$

This property guarantees that calculating the terms of the sequence z allows the value to be calculated using auxiliary sequences and only addition operations. These elements are given in the context P3-0. The contract can then be written in the form of a machine. P3-1 contains a single event, final, whose action is `r := z(`x_0`)`.

The method consists in refining the machine P3-1 into a machine P3-2 and introducing an iteration variable i covering the interval $0..x_0$ and variables for each sequence uu, vv, ww, tt, zz whose role is to store values of sequences to calculate $z(x_0)$. A new event is introduced to update the variables uu, vv, ww, tt, zz and i. The invariant expresses the link between the mathematical values of the sequences u, v, w, t, z and the values stored and calculated in the variables uu, vv, ww, tt, zz. The invariant is fairly easy to determine, but we need more relationships between the different sequences, so we come back to those sequences and give closed expressions that only mention the index.

Propery 2 *(Properties of sequences u,v,w,t)*

- $\forall k \in \mathbb{N} : v_k = 3 * k * k$
- $\forall k \in \mathbb{N} : w_k = 3 * k + 1$
- $\forall k \in \mathbb{N} : u_k = k$
- $\forall k \in \mathbb{N} : t_k = 6 * k + 3$

The properties are proved in the context P3-0 and will be used in the refinement of the machine P3-2. We introduce variables that point to the elements of the sequences that are sufficient for the computation. The refinement of P3-2 into P3-3 amounts to adding a variable for each sequence cu, cv, cw, ct, cz and these variables verify the following invariant property.

$@inv1\ i \in 0..x0$
$@inv2\ cv = vv(i)$
$@inv3\ cw = ww(i)$
$@inv4\ cz = zz(i)$
$@inv5\ ct = tt(i)$
$@inv6\ cu = uu(i)$

In addition, the events final and step are refined by making their guards *computable*. An expression of the form $x0 \in dom(zz)$ or $x0 \notin dom(zz)$ is difficult to translate efficiently into an algorithmic language. Thus, the new guard $i < x0$ implies $x0 \notin dom(zz)$ and the new guard $i = x0$ implies $x0 \in dom(zz)$. The resulting machine is therefore more deterministic and closer to an algorithmic (computable) expression. Proof obligations are discharged automatically.

We finish by hiding the variables uu, vv, ww, tt, zz in a refinement machine P3-4 of the machine P3-3. It remains to use the property of sequences that we have proved in the context. The proof effort made in the context P3-0 pays off when it comes to expressing the invariant properties constituting the loop invariant of the iterative algorithm produced from this machine.

The variable cu is useless, since $cu = i$. We have translated the machine P3-4 into the annotated C program of Listing 1.1, automatically verified by Frama-C. We obtained a cross-check of the algorithm derived from the machine P3-4.

Listing 1.1: ACSL power3.c

```
/*@  requires  0 <= x;
     ensures \result == x*x*x;
     assigns \nothing;
*/
int power3(int x)
{int  r, cz,cv,cu,cw,ct, i;
  cz=0;cv=0;cw=1;ct=3;cu=0;i=0;
       /*@
        @ loop invariant 0 <= i && i  <= x; // inv1
        @ loop invariant ct == 6*i +3; // inv5
        @ loop invariant cv== 3*i*i; // inv2
        @ loop invariant cw == 3*i+1; // inv3
        @ loop invariant cz == i*i*i; // inv4
        @ loop invariant cu == i; // inv6
        @ loop assigns ct,cz,i,cv,cw,cu; // specific Frama-c
        @ loop variant x-i; // variant in Event-B
*/
  while (i<x)
        {
          cz=cz+cv+cw;
          cv=cv+ct;
          ct=ct+6;
          cw=cw+3;
```

```
            cu = cu+1;
            i=i+1;}
  r=cz;return(r);
}
```

The archive bb-power3 contains all the machines used to develop this algorithm. The useless variable cu can be removed from the machine P3-4 by one more refinement step, which is safe by construction.

1.3.2. *Problem 3: Searching for a value in an array*

The problem is to find an occurrence of a value x in an array t of length n. There are no constraints on the array or the search technique.

variables t, n, x, r

requires
$$\begin{bmatrix} n_0 \in \mathbb{N} \\ x_0 \in \mathbb{V} \\ t_0 \in 1..n_0 \rightarrow \mathbb{V} \end{bmatrix}$$

ensures
$$\begin{bmatrix} t_f = t_0 \wedge n_f = n_0 \wedge x_f = x_0 \\ x_f \notin ran(t_f) \Leftrightarrow r_f = (0, FALSE) \\ x_f \in ran(t_f) \Leftrightarrow ((prj2(r_f) = TRUE) \wedge (t_f(prj1(r_f)) = x_f)) \end{bmatrix}$$

This problem must be reformulated in the form of a sequence u that expresses, for each $i \in 0..n$, whether the array t contains the value x between the indices 1 and i. As soon as the value is found for the first time at index i, we have $u(i) = (i, TRUE)$ and $u(j) = u(i)$ for all $j \in i..n$: the value found is i. If the value x is not in the array, then the sequence u is constantly equal to the pair $(0, FALSE)$. $prj1(p)$ (resp. $prj2(p)$) returns the first (resp. second) component of the pair p.

We define the sequence u in the context S-0 (see Figure 1.3) and use the same methodology. We now simplify the writing by defining x, t, n as constants, since they do not change. The problem is to find whether the value x occurs in the array t, and only r is a variable. The context S-0 contains the definitions of the data t, n, x and the axioms defining the sequence u whose value $u(n)$ is the expected solution.

The machine S-1 expresses that the desired value is $u(n_0)$ and in fact expresses the contract for this problem. As in the previous cases, the machine S-1 is refined into a machine S-2 which introduces the variables i and uu defining the inductive

CONTEXT $S-0$
SETS V
CONSTANTS $n\ t\ x\ u$
AXIOMS
$@axm1\ n \in \mathbb{N}$
$@axm2\ t \in 1..n \rightarrow V$
$@axm3\ x \in V$
$@axm4\ u \in 0..n \rightarrow \mathbb{Z} \times BOOL$
$@axm5\ u(0) = 0 \mapsto FALSE$
$@axm6\ \forall i\,.\, i \in 0..n-1 \land t(i+1) = x \land (\forall k.\, k \in 1..i \Rightarrow t(k) \neq x) \Rightarrow u(i+1) = i+1 \mapsto TRUE$
$@axm7\ \forall i\,.\, i \in 0..n-1 \land t(i+1) \neq x \Rightarrow u(i+1) = u(i)$
$@axm8\ \forall i\,.\, i \in 0..n-1 \land t(i+1) = x \land (\exists k.\, k \in 1..i \land t(k) = x) \Rightarrow u(i+1) = u(i)$
$@axm9\ \forall A.\, A \subseteq 0..n \land 0 \in A \land \{i+1 | i \in A \land i < n\} \subseteq A \Rightarrow 0..n \subseteq A$
$theorem\ @axm10\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = FALSE \Rightarrow prj1(u(i)) = 0)$
$theorem\ @axm11\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = TRUE \Rightarrow prj1(u(i)) \in 1..i)$
$theorem\ @axm15\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = TRUE \Rightarrow t(prj1(u(i))) = x)$
$theorem\ @axm17\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = FALSE \land i \neq 0 \Rightarrow u(i) = u(0))$
$theorem\ @axm16\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = FALSE \land i \neq 0 \Rightarrow t(i) \neq x)$
$theorem\ @axm14\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = TRUE \Rightarrow prj1(u(i)) \in 1..i)$
$theorem\ @axm12\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = TRUE \Rightarrow (\exists k.\, k \in 1..i \land t(k) = x))$
$theorem\ @axm13\ \forall i.\, i \in 0..n \Rightarrow (prj2(u(i)) = FALSE \Rightarrow (\forall k.\, k \in 1..i \Rightarrow t(k) \neq x))$
$theorem\ @axm22\ \forall i.\, i \in 1..n \land (\forall k.\, k \in 1..i \Rightarrow t(k) \neq x) \Rightarrow (prj2(u(i)) = FALSE \land prj1(u(i)) = 0)$

Figure 1.3. *Context S-0*

calculation scheme and introducing two events step1 and step2; unlike the previous cases, these events simulate a conditional instruction according to the axioms defining u. The process continues with the introduction of the value of the sequence uu useful for the calculation, i.e., cu, in the machine S-3. The machine S-4 hides the variable uu and produces a fairly classic algorithm.

$r := (0, FALSE) \parallel k := 0$
$\parallel cuu := (0, FALSE)$
while $k < n$ do
  if $t(k+1) = x$ then
    if $prj2(cuu) = FALSE$ then
      $k := k + 1$
      $\parallel cuu := (k+1, TRUE)$
    else
      $k := k + 1$
    fi
  else
    $k := k + 1$
  fi
od;
$r := cuu$;

$r := (0, FALSE) \parallel k := 0$
$\parallel cuu := (0, FALSE)$
while $k < n$ do
  if $t(k+1) = x \wedge prj2(cuu) = FALSE$ then
    $cuu := (k+1, TRUE)$
  fi;
  $k := k + 1$;
od;
$r := cuu$;

The db-searching archive contains all the machines used to develop this algorithm. Note that we apply the merging of events twice to obtain two conditional statements: first step1 and step3, then mergingif(step1,step3) and step2; finally, we apply the iteration rule. A simple rewriting of the algorithm simplifies the body of the iteration (right).

1.3.3. *Problem 4: Computing a primitive recursive function*

Primitive recursive functions correspond to bounded iterations, and a primitive recursive function is constructed from a scheme using two primitive recursive functions to define a third function. Examples of such functions are addition, multiplication, division, primality, ... In a way, it is the general case restricted to bounded iteration.

The problem is to calculate the function F defined using two other functions G and H which are calculated by algorithms. F is defined by the following equations: for all natural numbers x, y, $\begin{cases} F(x,0) = G(x) \\ F(x,y+1) = H(x,y,F(x,y)) \end{cases}$

variables x, n, r

---

services
$H \in \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
$G \in \mathbb{N} \to \mathbb{N}$
$F \in \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
$F = PRIMREC(G, H)$

---

requires $\begin{pmatrix} x_0 \in \mathbb{N} \\ n_0 \in \mathbb{N} \end{pmatrix}$

ensures $\begin{pmatrix} x_f = x_0 \wedge n_f = n_0 \\ r_f = F(x_0, n_0) \end{pmatrix}$

The function F is defined using G and H, which are themselves defined by the same computability schemes. For a fixed x, the function $k \mapsto F(x, k)$ is a sequence defined by induction on k; the variable cv stores its current value $F(x, k)$. We obtain the following algorithm.

$r :\in \mathbb{N} \parallel k := 0 \parallel cv := G(x)$
while $k < n$ do
$k := k + 1 \parallel$
$cv := H(x, k, cv)$
od;
$r := cv$

The eb-primitiverecursive archive contains all the machines used to develop this algorithm.

Jean-Raymond Abrial (**?**, Chapter 15) gives a list of sequential algorithms built using the two transformations and provides the Rodin[1]archives. It is important to note that Jean-Raymond Abrial's examples begin with a machine with an event final modelling the observation of a procedure call corresponding to the problem solved, and also an event progress whose status *anticipated* means that events will appear later to model one or more future loops. The event progress models the implicit and hidden presence of intermediate computations which must respect the invariant of the abstraction level. The event *conserves* or *preserves* the calculation and its invariant; we sometimes call it keep as opposed to an event which is always present called skip. Finally, it is quite clear that the translation still requires an intervention to produce an executable program, as we have shown with Listing 1.1, and this allows us to check that the translation is correct. We therefore use Frama-C to verify that the resulting algorithm is partially correct with respect to the contract. The loop invariant is derived from the Event-Bdevelopment. Jean-Raymond Abrial uses Hoare triples to express the specification of the problem to be solved and we prefer to use contracts that are available in programming languages such as ACSL/Frama-C or Spec#/Boogie. Readers may ask about the soundness of the transformations: no proof of correctness has been written yet, and this proof remains future work.

1. The link `https://web-archive.southampton.ac.uk/deploy-eprints.ecs.soton.ac.uk/122/index.html` gives a list of sequential program developments with a tutorial detailing how to refine and what to transform.

1.4. Design of a Recursive Sequential Algorithm

In this section, we apply the simple idea of the diagram in Fig. 1.1 to develop a *recursive* algorithm. We promote one-shot refinement. In the previous section, we refined as long as necessary, i.e., until we obtained a set of events corresponding to observed computations. We have produced the EB2RC plugin (**?**), which implements this idea. In the diagram of Fig. 1.4, expressions such as start, callrec and end mean that control is at start (resp. callrec, end), when start (resp. callrec, end) is true. The expression callrec indicates that control is after the call to the procedure P in question; this call is observed by an event named $P(x-1, tr) : x \neq 0$.

1.4.1. *The "Call as Event" Idea*

The refinement-based design of iterative sequential algorithms uses a sequence of values in a domain $\mathcal{D}$. The computation process is based on recording the values of the sequence. In the case of the call-as-event paradigm, the pattern is based on the link between the occurrence of an event and a call of a function (or procedure or method) satisfying the pre-condition and post-condition respectively at the call point and the return point. The context C0 defines the sequence of values and the definition of the sequence is used as a guide for the shape of events. The definition of the sequence is reformulated by a diagram which simulates the different cases when the procedure under development, namely $P(x, r)$, is called.

The diagram in Fig. 1.4 is derived from the Event-B machine ALGOREC and is a finite state diagram. We use special names for events in the diagram: P(0,r): $x = 0$ stands for the event observed, when the procedure P(x,r) is called with x=0; P(x-1,tr): $x \neq 0$ models the observation of the recursive call of P; P(x,r): $x \neq 0$ stands for the event observed when the procedure P(x,r) is called with $x \neq 0$. P(0,r): $x = 0$ and P(x,r): $x \neq 0$ refine the event computing which is observed when the procedure P is called. The red arrows express a case analysis and the blue arrows model events. The diagram provides a liveness argument very close to the proof lattices of Owicki and Lamport (**?**).

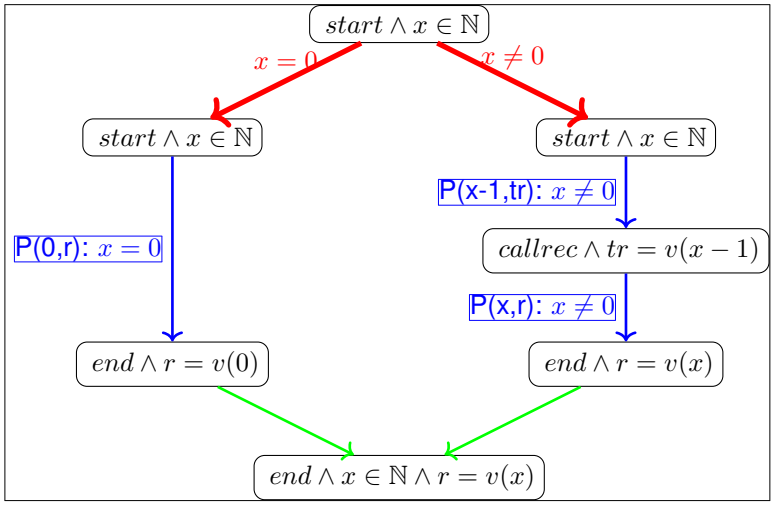


Figure 1.4. *Organisation of the computation in a recursive solution using assertion diagram*

MACHINE $ALGOREC$
REFINES $PREPOST$
SEES $C0$
VARIABLES
r, pc, tr
INVARIANTS
$art : pc \in L$
$inv1 : tr \in D$
$inv2 : pc = callrec \Rightarrow tr = v(x-1)$
$inv3 : pc = end \Rightarrow r = v(x)$

CONTEXT $C0$
SETS D, L
CONSTANTS $v, x, d0, f, start, end, callrec$
AXIOMS
$ax1 : x \in \mathbb{N}$
$ax2 : v \in \mathbb{N} \rightarrow D$
$ax3 : partition(L, \{start\}, \{end\}, \{callrec\})$
$\ldots$

The refinement machine is organised with respect to the inductive definition using a control variable pc. The variable pc controls the different steps of the computations simulated by the events. The invariant is derived directly from the definitions of the intermediate values. Proof obligations are simple to discharge. It remains to prove that the values of the sequence v correspond to the required values given in the post-condition.
The machine ALGOREC and the context C0 are given in incomplete form, but the idea is to show that the diagram is linked to the analysis carried out during the computation of the sequence v using a recursive process.

Event P(0,r):x=0	Event P(x-1,tr):x≠0	Event P(x,r):x≠0
REFINES computing	WHEN	REFINES computing
WHEN	$grd1 : pc = start$	WHEN
$grd1 : x = 0$	$grd2 : x \neq 0$	$grd1 : pc = callrec$
$grd2 : pc = start$	THEN	THEN
THEN	$act1 : tr := v(x-1)$	$act1 : r := f(tr)$
$act1 : r := d0$	$act2 : pc := callrec$	$act2 : pc := end$
$act2 : pc := end$	END	END
END		

The refinement machine models computations following two cases according to Fig. 1.4. The first case ($x = 0$) is the path on the left part of the diagram and the second case ($x \neq 0$) is the path on the right part.

The first path ($x = 0$) is a three-step path and is labelled by the condition $x = 0$ and the event P(0,r):x=0. The event P(0,r):x=0 assigns the value $d0$ to r according to the definition of $v(0)$. P(0,r):x=0 refines the event computing in the machine PREPOST. The third step is an implication leading to the postcondition.

The second path ($x \neq 0$) is *four steps* long, and the event P(x-1,tr):$x \neq 0$ simulates the recursive call of the same procedure. Finally, the event P(x,r):$x \neq 0$ refines the event computing. The call as event paradigm is applied, when one considers that one event defines the specification of the recursive call and the user gives the name of the call to indicate that the event should be translated into a call. The EB2RC plugin (**?**) generates automatically a C-like program. The machine ALGOREC is simple to check. Proof obligations are simple, because the recursive call hides the previous values stored in the variable uu of the iterative paradigm.

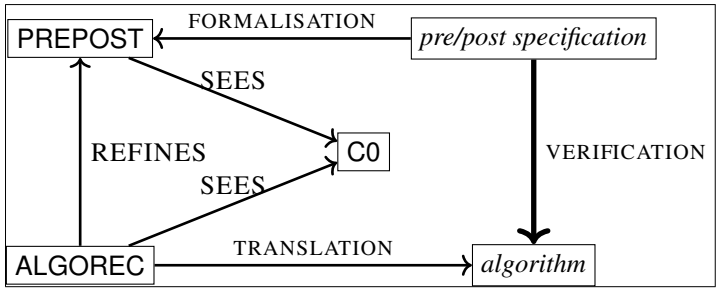


Figure 1.5. *The recursive pattern*

The recursive pattern is linked to a diagram which helps to structure the solution. We have labelled arrows by guards or by events. The diagram helps to structure the

analysis based on the inductive definitions. Following this pattern, we have developed the EB2RC plugin, based on the identification of three possible events. When a pre/-post specification is stated, the program to build can be expressed by a simple event expressing the relationship between input and output and it provides a way to express a pre/post specification as an event. The first machine contains this event, called computing. The refinement-based process requires an idea for introducing more concrete events. A very simple and powerful way to refine is to introduce a more concrete model which is based on an inductive definition of outputs with respect to the input.

A first consequence is that the concrete model ALGOREC contains events which compute the same function but corresponding to a recursive call expressed as events (Event rec%PROC(h(x),y)%P(y)).

Event e
where
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x)$
then
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x)$
end

The event Event rec%PROC(h(x),y)%P(y) simply simulates the recursive call of the same function and this expression makes the proofs easier. The invariant is defined in a simpler way by analysing the inductive structure and a control variable is introduced for structuring the inductive computation. We have identified three possible events to use in the concrete model:

Event
rec%PROC(h(x),y)%P(y)
any y
where
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x, y)$
then
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x, y)$
end

Event
call%APROC(h(x),y)%P(y)
any y
where
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x, y)$
then
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x, y)$
end

Each event has a name that follows one of three formats, and the user assigns this name. This convention facilitates the translation of the refinement machine into an algorithm. If the name begins with the character string "rec%…", it corresponds to a recursive call to the current function, and the name ends with the name of the function and the case. Rodin allows any string of characters to be used as an event name. A second case is an event of the form "call%APROC(h(x),y)%P(y)", which indicates that another function or procedure, called APROC, must be used and should satisfy the local contract. The name of this function or procedure is indicated by the string of characters following the % symbol. In other cases, the event is translated into an algorithmic expression. The user plays a central role, and the tool checks the

consistency of the machine-produced parts. The % symbol is used, but another special symbol may be used instead, such as $ or @; the examples below use @.

1.4.2. *Applying the call as event technique*

We will illustrate this method of designing recursive programs using a few relevant examples.

1.4.2.1. *Problem 5: Computing the square ($\lambda x.x^2$)*

Applying the recursive pattern is made easier by the first steps of the iterative pattern. In fact, the context C0 and the machine PREPOST are the starting points of the iterative pattern as well as the recursive pattern. We use the computation of the function x^2 and obtain the following refinement of PREPOST. Fig. 1.4 is the diagram analysing the way to solve the computation of the value of $u(x)$ following the call-as-event paradigm.

MACHINE $square$
REFINES $specquare$
SEES $control0$

VARIABLES $r\ l\ tr$

INVARIANTS
$@inv1\ r \in \mathbb{N}$
$@inv2\ l = end \Rightarrow r = n * n$
$@inv3\ l = call \Rightarrow n \neq 0$
$@inv4\ l = call$
$\Rightarrow tr = (n-1) * (n-1)$
$@inv5\ l \in C$
$@inv6\ tr \in \mathbb{N}$
$@inv7\ l = end \Rightarrow r = n * n$
$@inv8\ l = end \wedge n \neq 0$
$\Rightarrow tr = (n-1) * (n-1)$
$\wedge\ r = tr + 2 * (n-1) + 1$
$theorem\ @inv9\ l = call$
$\Rightarrow n * n = tr + 2 * (n-1) + 1$

EVENTS
EVENT $INITIALISATION$
then
$@act1\ r := 0$
$@act2\ l := start$
$@act3\ tr :\in \mathbb{N}$
end

EVENT $square(n;r)@n = 0$
REFINES $square(n;r)$
where
$@grd1\ l = start$
$@grd2\ n = 0$
then
$@act1\ l := end$
$@act2\ r := 0$
end

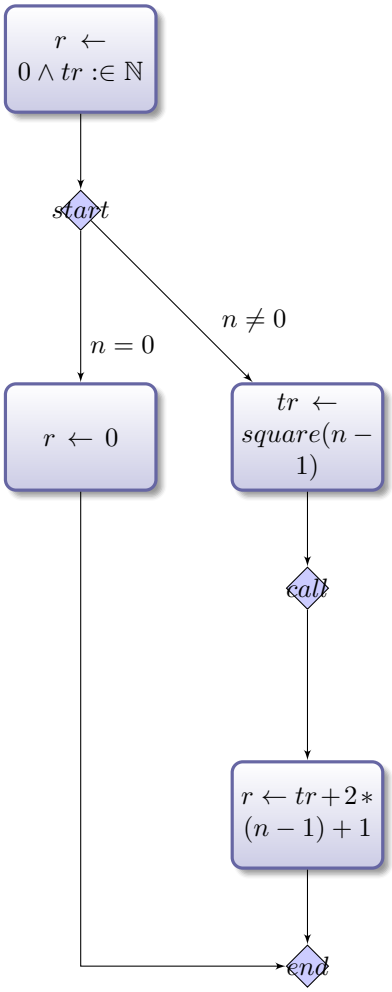
The variable l models the control in the diagram. We introduce control points corresponding to assertions in the labels of the diagram as $C = \{start, end, call\}$. Three

events are defined and the invariant is written very easily and proofs are derived automatically. The event rec@square(n-1;tr)@n≠0 is the key event modelling the recursive call. In the current example, we have modified the machine by using directly the fact that $v(n) = n * n$; normally, we would use the sequence following the recursive pattern and then derive the theorem $v(n) = n * n$.

EVENT $square(n; r)@n/ = 0$ REFINES $square(n; r)$
where
$@grd1\ l = call$
then
$@act1\ r := tr + 2 * (n - 1) + 1$
$@act2\ l := end$
end

EVENT $rec@square(n - 1; tr)@n \neq 0$
where
$@grd1\ l = start$
$@grd2\ n \neq 0$
then
$@act1\ l := call$
$@act2\ tr := (n - 1) * (n - 1)$
end
end

Proofs are simpler and invariants are easier to extract from the inductive definitions. From the contexts and the machines, by following the rules and by choosing a name for the elements that allow us to produce an algorithm using the EB2RC plugin, we obtain a recursive algorithm that meets the problem specification, together with a diagram that shows the different steps of this algorithm.



The diagram is produced with tikz and has annotations defined by this list. The algorithm produced is given below and is very simple to produce from the model

```
procedure square(n; r)
begin
r ← 0
tr :∈ ℕ
if n = 0
      r ← 0
elsif n ≠ 0
      tr ← square(n − 1)
      r ← tr + 2 ∗ (n − 1) + 1
endif
end
```

The invariant states simple and obvious properties related to control points. Theorem 9 is particularly worth examining. It is easy to derive because it corresponds to the recursive call. All the calls and all the details are swept under the carpet, leaving only the last call. This shows the importance and interest of recursion.
The diagram illustrates one functionality of the EB2RC tool.

The technique is based on the main invariant inv8, which uses the classical identity $\forall n \in \mathbb{N}.n \neq 0 \Rightarrow n^2 = (n-1)^2 + 2 * (n-1) + 1$

MACHINE $square\ //\ square(n;r)$
REFINES $specquare$ SEES $control0$

VARIABLES $r\ l\ tr$

INVARIANTS
$@inv1\ r \in \mathbb{N}$
$@inv2\ l = end \Rightarrow r = n * n$
$@inv3\ l = call \Rightarrow n \neq 0$
$@inv4\ l = call \Rightarrow tr = (n-1) * (n-1)$
$@inv5\ l \in C$
$@inv6\ tr \in \mathbb{N}$
$@inv7\ l = end \Rightarrow r = n * n$
$@inv8\ l = end \wedge n \neq 0$
$\quad \Rightarrow tr = (n-1) * (n-1) \wedge r = tr + 2 * (n-1) + 1$
$theorem\ @inv9\ l = call \Rightarrow n * n = tr + 2 * (n-1) + 1$

1.4.2.2. *Problem 6: Binary search in an array*

We solve the problem of *searching for a value in a sorted array*. The input parameters of the *binsearch* procedure are: a sorted array t; the bounds of the array within which the algorithm should search (lo and hi); and the value for which the algorithm should search (val). Output parameters are $result$ and a boolean flag ok that indicates if $t(result) = val$. The pre- and postconditions are as follows:

contract $binsearch(t, val, lo, hi, ok, result)$

$$\text{requires} \begin{pmatrix} t \in 0..t.Length - 1 \longrightarrow \mathbb{N} \\ \forall k.k \in lo..hi - 1 \Rightarrow t(k) \leq t(k+1) \\ val \in \mathbb{N} \\ lo, hi \in 0..t.Length - 1 \\ lo \leq hi \end{pmatrix}$$

$$\text{ensures} \begin{pmatrix} ok = TRUE \Rightarrow result \in lo..hi \wedge t(result) = val \\ ok = FALSE \Rightarrow (\forall i.i \in lo..hi \Rightarrow t(i) \neq val) \end{pmatrix}$$

The array t is sorted with respect to the ordering over integers and a simple inductive analysis is applied, leading to a binary search strategy.

The specification is first expressed by two events corresponding to the two possible cases: either a key exists in the array t containing the value val, or there is no such

key. These two events correspond to the two possible resulting *calls* to the procedure $binsearch(t, val, lo, hi; ok, result)$:

– Event find is $binsearch(t, val, lo, hi; ok, result)$ where $ok = TRUE$

– Event fail is $binsearch(t, val, lo, hi; ok, result)$ where $ok = FALSE$

These two events form the machine called *binsearch1*, which is refined to obtain *binsearch2* (corresponding to PROCESS of Figure 1.10). In addition to these events, the events of this refined machine use a new control label, l, which *simulates* how the binary search is achieved.

EVENT $find$
  any j
  where
  $@grd1\ j \in lo..hi$
  $@grd2\ t(j) = key$
  then
  $@act1\ ok := TRUE$
  $@act2\ i := j$
end

EVENT $fail$
  where
  $@grd1\ \forall k\ k \in lo..hi \Rightarrow t(k) \neq key$
  then
  $@act1\ ok := FALSE$
end

The refinement of binsearch1 is interesting because it shows an invariant derived from the properties of the decomposition following the analysis of the search process. The invariant explicitly considers the role of the index *middle* and it is a clear statement of the property. We say that the recursion sweeps the dust under the carpet, which makes the solution simpler to write and to read.

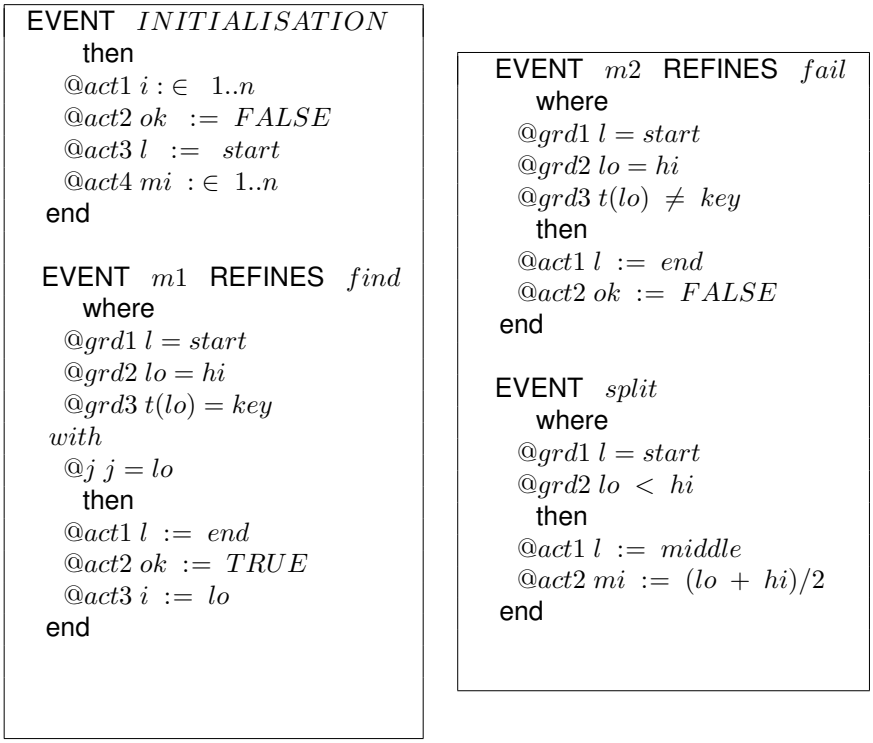
MACHINE $binsearch2$
REFINES $binsearch1$
SEES $binsearch0$

VARIABLES $i\ ok\ l\ mi$

INVARIANTS
$@inv1\ i \in 1..n$
$@inv2\ l \in LOC$
$@inv3\ dom(t) = 1..n$
$@inv4\ mi \in 1..n$
$@inv5\ l = middle$
$\Rightarrow\ lo < hi \wedge mi \in lo..hi$
$@inv6\ l = middle \wedge key < t(mi)$
$\Rightarrow\ (\forall\ k\ k \in mi..hi \Rightarrow t(k) \neq key)$
$@inv7\ l = middle \wedge key > t(mi)$
$\Rightarrow\ (\forall\ k\ k \in lo..mi \Rightarrow t(k) \neq key)$
$@inv8\ l = end \wedge ok = TRUE$
$\Rightarrow\ i \in lo..hi \wedge t(i) = key$
$@inv9\ l = end \wedge ok = FALSE$
$\Rightarrow\ (\forall\ k\ k \in lo..hi \Rightarrow t(k) \neq key)$
$theorem\ @inv10\ lo..hi \subseteq 1..n$
$theorem\ @inv11$
$(\exists\ j\ l = middle \wedge j \in mi+1..h$
$\wedge\ key > t(mi) \wedge t(j) = key \wedge mi+1 \leq hi)$

$\Rightarrow\ (mi+1 \leq hi)$

The first event is the event INITIALISATION, the two events m1 and m2 corresponding to the case $lo = hi$ and the event split which is corresponding to the case $lo < hi$. The events follow the case analysis.



The event split directs the analysis and divides the exploration space into three parts. In Figure 1.9, the event m3 considers the case where the index mi is the one where the searched value is found. Then the other two events correspond to the segment $mi + 1..hi$ and are in fact recursive calls to the procedure under construction. These two events refine find and fail respectively. We need to add two more events corresponding to the segment $lo..mi - 1$ to complete the model.

A textual representation of the binary search algorithm is constructed by EB2RC (Fig. 1.7); the diagram of Fig. 1.6 is also produced by the tool. The generated algorithm has been compared to an algorithm written by hand. The two algorithms are identical up to a slight reformatting.

The proof effort of our refinement approach for the Binary Search case study is illustrated in Table 1.1. The first abstract model is proved automatically and the second concrete model is automatic in 89 % of its proof obligations.

The integrated development framework takes this into consideration. As shown in Fig. 1.10, we suggest translating every recursive algorithm ALGORITHM into a partially annotated and iterative OPTIMISED ALGORITHM to be verified within the Spec# Programming System. In (**?**), we have proposed and proved a sound translation procedure from ALGORITHM to OPTIMISED ALGORITHM to perform

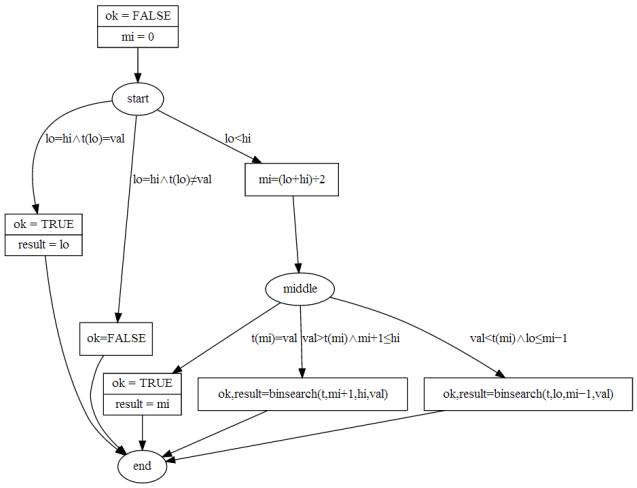


Figure 1.6. *Visualized Representation of the Binary Search Algorithm*

Model	Total	Auto	Manual	Reviewed	Undischarged	% auto
binsearch1	5	5	0	0	0	100 %
binsearch2	71	63	8	0	0	89 %

Table 1.1. *Proof effort of our refinement approach for the binary search case study*

this task. For example, the iterative version of the binary search algorithm in Spec# is shown in Fig. 1.8.

By sending this program to Spec#, Spec# reports the program as verified with respect to partial correctness: the termination of the loop is not checked, and indeed the loop does not progress when $lo < hi$ and no branch of its body applies (for instance $lo = 0$, $hi = 1$ and $val < t[0]$); the recursive algorithm handles this case by returning $ok = FALSE$, and the iterative version needs an additional exit condition. No user interaction is required in this verification as all assertions required (preconditions, postconditions and loop invariants) have been generated as part of the refinement and transformation of the initial abstract specification into the final iterative algorithm.

1.4.3. *Comments on the call as event idea*

In the example of Section 1.4.2.1, we do not use an event like call%APROC(h(x),y)%P(y), which models a call to another procedure or function.

Recursive Algorithm binsearch(t,lo,hi,val;ok,result) generated by EB2RC
begin
$ok := FALSE; mi := 0;$
if $lo = hi \wedge t(lo) = val$ then
  $ok := TRUE;$
  $result := lo;$
elseif $lo = hi \wedge t(lo) \neq val$ then
  $ok := FALSE;$
elseif $lo < hi$ then
  $mi := (lo + hi) \div 2;$
  if $t(mi) = val$ then
    $ok := TRUE;$
    $result := mi;$
  elseif $val > t(mi) \wedge mi + 1 \leq hi$ then
    $ok, result := binsearch(t, mi + 1, hi, val);$
  elseif $val < t(mi) \wedge lo \leq mi - 1$ then
    $ok, result := binsearch(t, lo, mi - 1, val);$
  end
end

Figure 1.7. *Recursive Algorithm binsearch(t,lo,hi,val;ok,result) generated by* EB2RC

For instance, when a sorting algorithm is developed, one may need an auxiliary operation for scanning a list of values to get the index of the minimum. It means that we have a way to define a library of models and to use correct-by-construction procedures or functions. In (**?**), we detail the tool and the way to define a library of *correct-by-construction programs*. The EB2RC plugin is used on this project and we obtain two files: one containing the algorithm and another containing the diagram built from the Event-Bmodel.

1.5. Final comments

We have presented the use of the Event-Blanguage in the derivation of sequential programs or algorithms. The first technique in fact uses the sequences of values leading to a given term constituting the sought-after solution. Thus the calculation of the sum or the cube is carried out by defining a sequence one of whose terms is the value of the sum or the cube. Since refinement is a very general relation on very general models, we must set ourselves a refinement discipline. The first technique invites us to play with the model variables and to reduce the variables to those which are exclusively used for the calculation. In this way, we can better understand the notions

```
class BS {
 int BinarySearch(int[] t, int val, int lo, int hi, bool ok)
  requires 0 <= lo && lo < t.Length && 0 <= hi && hi < t.Length;
  requires lo <= hi && 0 < t.Length;
  requires forall{int i in(0:t.Length),int j in (i:t.Length);t[i]<=t[j]};
  ensures -1 <= result && result < t.Length;
  ensures (0 <= result && result < t.Length)==> t[result] == val;
  ensures result == -1 ==> forall {int i in (lo..hi); t[i] != val};
 { int mi = (lo + hi) / 2;
   while (!((lo == hi && t[lo] == val) || ( lo == hi && t[lo] != val)
                || (lo < hi && (mi == (lo + hi) /2) && t[mi] == val)))
    invariant 0 <= lo && lo < t.Length && 0 <= hi && hi < t.Length;
    invariant 0 <= mi && mi < t.Length;
    invariant (val < t[mi]) ==> forall {int i in (mi..hi); t[i] != val};
    invariant (val > t[mi]) ==> forall {int i in (lo..mi); t[i] != val};
   { mi = (lo + hi) / 2;
     if ((mi+1 <= hi) && (val > t[mi])) lo = mi +1;
     else if ((lo <= mi-1) && (val < t[mi])) hi = mi - 1;
   }
   if ((lo == hi) && (t[lo] == val)) {ok = true; return lo;}
   else{
    if ((lo == hi) && (t[lo] != val)) {ok = false; return -1;}
    else if ((lo < hi) && (t[mi] == val)) {ok = true; return mi;}
    else {ok = false; return -1;}
   }
 }
}
```

Figure 1.8. *Binary Search Spec# program corresponding to the generated iterative procedure.*

of model variable or ghost variable used in certain proof tools. In our case, we introduce them in the abstract models and then hide them in the concrete models. They make invariants easier to state and to prove. The second technique is simpler because it relies on inductive definitions that are interpreted in the recursive paradigm. It saves refinement steps, since the development is carried out in a single refinement step. The resulting program is recursive; the recursion can then be eliminated, and this is relatively easy to produce using our event conventions. In this case, we noted that the proofs were relatively simpler to derive. Both techniques rely on experimental plugins but could be combined. Figure 1.10 illustrates an environment for coordinating the various techniques for relatively complex sequential systems; it remains to develop a new formal IDE integrating these techniques, close to the diagram of Figure 1.10. Naturally, the development of concurrent, parallel or distributed algorithms or programs is a challenge to be taken up, and the chapter entitled *Design of Correct by Construction Distributed Algorithms* will provide some ideas.

EVENT $m3$ REFINES $find$
where
$@grd1\ l = middle$
$@grd3\ t(mi) = key$
$with$
$@j\ j = mi$
then
$@act1\ l := end$
$@act2\ ok := TRUE$
$@act3\ i := mi$
end

EVENT $rec@search(t, val, mi + 1, hi, ok, result)@ok = TRUE$
REFINES $find$
any j
where
$@grd1\ l = middle$
$@grd2\ key > t(mi)$
$@grd3\ j \in mi + 1..hi$
$@grd4\ t(j) = key$
$@grd5\ mi + 1 \leq hi$
then
$@act1\ i := j$
$@act2\ ok := TRUE$
end

EVENT $rec@search(t, val, mi + 1, hi, ok, result)@ok = FALSE$
REFINES $fail$
where
$@grd1\ l = middle$
$@grd2\ key > t(mi)$
$@grd4\ \forall\ j\ \ j \in mi + 1..hi \Rightarrow t(j) \neq key$
$@grd5\ mi + 1 \leq hi$
then
$@act3\ l := end$
$@act2\ ok := FALSE$
end

Figure 1.9. *Events for recursive calls*

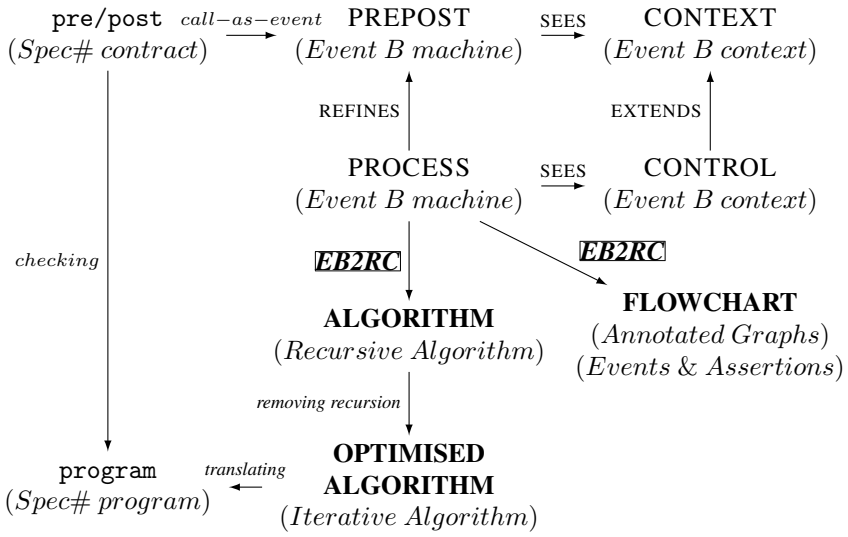


Figure 1.10. *An overview of our integrated development framework to combine program refinement with program verification ((?))*